

Expressing, Managing, and Validating User Stories: Experiences from the Market

Carlo Pecchia, Marco Trincardi and Pietro Di Bello

Abstract Agile methodologies for software development favor customer involvement and thus a rapid feedback cycle on realized product increments. Such involvement is implemented in the activities around requirements (elicitation, analysis, development, management, change, validation), which in turn are often sustained by—and expressed in—“user story” format. This paper aims to show our experience in developing software system representing functional requirements mainly with “user stories,” and capturing also nonfunctional requirements (e.g., availability, security) in demanding domains. This paper starts defining what a user story is, how we write and test it, and what are main differences compared to “traditional” documented requirements and use cases. Then, it focuses on techniques we use for splitting and grooming, and how we transform a linear backlog into a multidimensional Story Map that help us to manage size and complexity.

1 Introduction

In this paper, we want to shortly introduce our development process, then focusing on User Story (what they are and how we use them), and finally we describe main benefits we observed from our experience.

C. Pecchia (✉) · M. Trincardi · P. Di Bello
XPeppers srl, Trento, Italy
e-mail: carlo.pecchia@xpeppers.com

M. Trincardi
e-mail: marco.trincardi@xpeppers.com

P. Di Bello
e-mail: pietro.dibello@xpeppers.com

2 Background

In developing products (systems), we use the agile framework known as Scrum [1]: an iterative approach where increments of functionality are implemented at each iteration. Beside Scrum, we make full use of Extreme Programming practices [2]: pair programming, test driven development, continuous integration, frequent releases, and so on. Each iteration starts with a short planning phase where some “user needs”—in the form of “user story”—are picked, discussed, analyzed by developing team with the business representative (also known as product owner), and then included in the iteration for developing. At the end of each iteration, we show the added increments of functionality in a demo session, where the product owner can give feedback for approval and/or necessary refinements (note this is a suitable opportunity to refine and adjust end user requirements).

One of the key tools used in this process are *User Stories*.

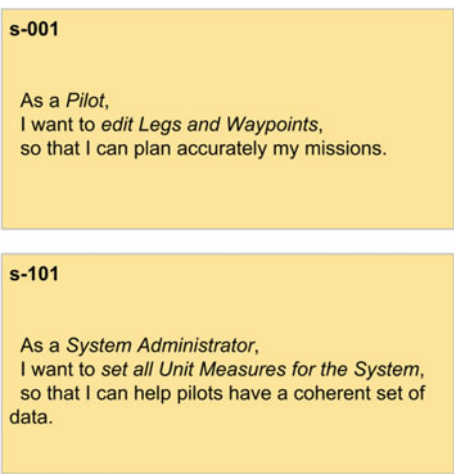
3 Introducing “User Stories”

A *User Story* can be defined as a short description of what end user does in order to achieve his functions when interacting with the system being developed. Generally, they consist in few sentences and are expressed in the business language. Figure 1 shows some examples.

A common practice, on writing user stories, consists of declaring explicitly the *who*, the *what*, and the *why* [3], as shown in Fig. 2.

A “role” (the “who”) in the system does something (the “what”) in order to achieve some business benefits (the “why”). However, other formats exist [4] that one is widely accepted and used mainly for its succinctness and *completeness*:

Fig. 1 Format examples



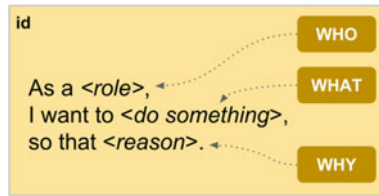


Fig. 2 General format

indeed such a format captures very essential aspects related to a functional requirement.

Some important aspects of user stories are listed below:

- the information carried by a single user story should be sufficient for its planning: can it be done in a single iteration or it should be split?
- a user story does not represent a formal requirement: it is a starting point for a real conversation with the end user (or his representative). Only with a real conversation, the development team can reach a (reasonable) shared understanding of what really are end user needs: a formal requirements document can be interpreted and understood in different ways by different people.
- being limited in scope, a single user story should be easily tested and verified.

3.1 *The Process “Induced” by User Stories*

To better understand the “process” enabled by user stories, let recall the 3C acronym [5]:

- **Card**—a conveniently short description of what functionality should be done from the end user standpoint.
- **Conversation**—a needed (face-to-face) conversation among development team about the user story.
- **Confirmation**—a common agreement on how to confirm that the user story is done.

Every single “C” is necessary in order to fully reach a shared understanding about what should be implemented and *how* it should be tested/verified. The agreement on the latter aspect (testing) is where generally important insights pop up; when we discuss on how to demonstrate the implementation of a single user story, we reason with (near) realistic data as examples. These discussions on “realistic examples” often permit us to find holes in story understanding, e.g., some external dependencies are not considered and data can present an unknown pattern.

Another important aspect, often misunderstood, is that “user stories” does not mean *lack of documentation*: is the *form* of the documentation that can vary.

When discussing a user story in front of a whiteboard often, we add some other information such as UI sketches, sample workflow diagrams. All those “low-fidelity” documentation can be easily grabbed with a camera and added to the user story as a reference, and it should be noted that it is far more expressive and cheap to produce than a long written documentation. Moreover, supporting documentation is written when needed, and not up-front.

3.2 *Evaluating User Story*

A widely accepted criterion applied in evaluating the goodness of a user story resides in the acronym INVEST [6]:

Independent

Each story should be completed (that means an increment of system functionalities can be done) independently from other stories; although some correlation, if not a strict dependency, may exist.

Negotiable

The scope of the user story can be adjusted when conversations—intended to clarify the user story—happen within development team.

Valuable

The story must deliver value to the intended business (directly or indirectly), if not one may consider to not developing it.

Estimable

The development team should be able to give an estimate, even a rough estimate, on the effort needed to implement the story. Otherwise, the business is not able to determine the effective cost/benefits ratio for the story.

Small

The story should be complemented in a single iteration; otherwise, it could be difficult to track progress or to be reasonably confident in having understood the whole story scope. As a rule of thumb, at least three stories should fit in an iteration.

Testable

When the user story is testable, the development team can confirm *when* it can be considered completed. Moreover, when those tests can be automated (that is a really desirable thing), they ensure that implementation of other user stories does not break those already done.

Applying a check on INVEST criteria, for each user story, gives to development team the confidence that the user stories “base” presents a sufficient level of quality that enables a smooth workflow. Of course, this “confidence” should not be guaranteed all up-front, but it can be achieved throughout iterations.

4 Testing User Stories

Together with each user story, we should describe its *acceptance criteria*: specific assertions that must be true for the user story to be considered “ready for the end user.” It should be noted here an important aspect: These criteria are intended to provide enough guidance to the development team when implementing the story rather than detailing each specific test case. No one will prevent use to add—if needed—tests that verifies all conditions for the user story.

We also use TDD [7] to design and test user story implementation, but this is a different type of testing that works at very low level. Tests (first and foremost those covering the aforementioned acceptance criteria) are written before a very line of code is written for the user story. Once an automated test is written, we start coding for the story just enough to make the test pass, then we start to see improvements in the code (this is also known as “red-green-refactor cycle”).

This approach gives us several benefits: (1) each story is covered by at least an automated test; (2) test automation enable us to run a regression test where other stories are implemented; and (3) we do not add functionalities not derived from acceptance criteria.

5 Differences from Requirements and from Use Cases

A general requirement is defined as [8]:

A condition or capability needed by a user to solve a problem or achieve an objective.

Beside this definition, we found also derived requirements and nontechnical requirements.

User stories, being less structured, cannot be assimilated with requirements. That said, when there are mandatory reasons to document other types of requirements, we have to rely on extensive written documentation. In our practice, we tend to keep it as simple as possible in order to not waste time in low value added artifacts and activities.

Use Cases are an extensively used approach to expressing and documenting the functionalities of a system [9]. There is no direct correlation between user stories and use cases, because they work on a different level of detail. If we would like to find a similarity, we could see user story as part of use case scenario, but the latter is not needed to define the former.

In our practice, we tend to keep use cases (when present) for specific parts of the systems where its complex behaviors is clearly and fully expressed.

6 Managing Complexity

At the beginning, the scope of a story is so wide that it is not possible for the development team to implement the story in a single iteration and—more important—the end users voice cannot at the moment give more details, but only express an high-level need. In such a case, we talk about an *Epic* rather than a user story (Fig. 3).

Also, when we have a bunch of related stories, we can, for convenience, call the whole bunch as a *Theme*.

When a story is too big for an easy and concise understanding from the development team, we found convenient to split the story (see the example on Fig. 4). The general idea is, of course, to give the maximum value to the end user.

Some useful criteria and advices in splitting the stories are presented below:

- if the story describes a (complex or long) workflow, we should consider to split the workflow in several stories (e.g., first the start and ending points, then the intermediate states).
- if the story uses the same data type but with different interfaces, we can split the story by the “basic” handling of the data type, and then other stories for each interface.
- if the story is mainly related to nonfunctional specification (e.g., performance), we can split the story in a “simple” working version and then another one focusing on performance improvements.

Fig. 3 Epic samples

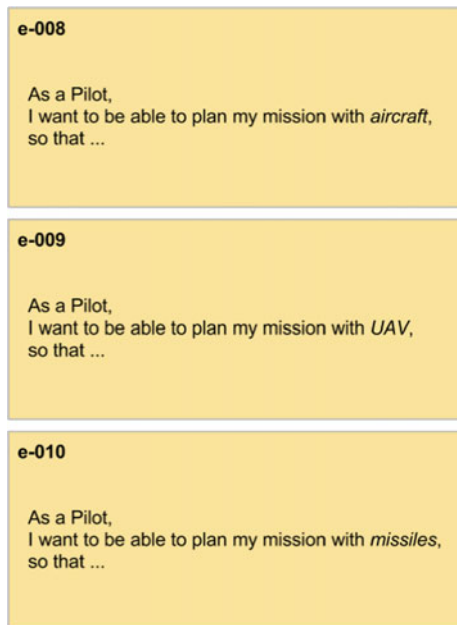
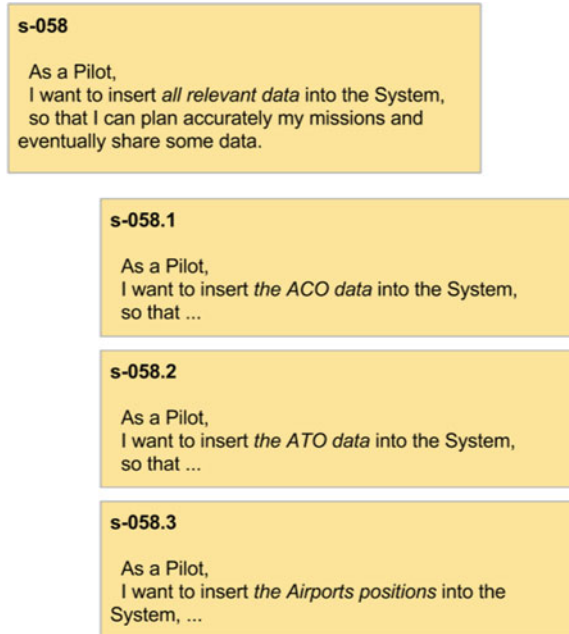


Fig. 4 Splitting sample

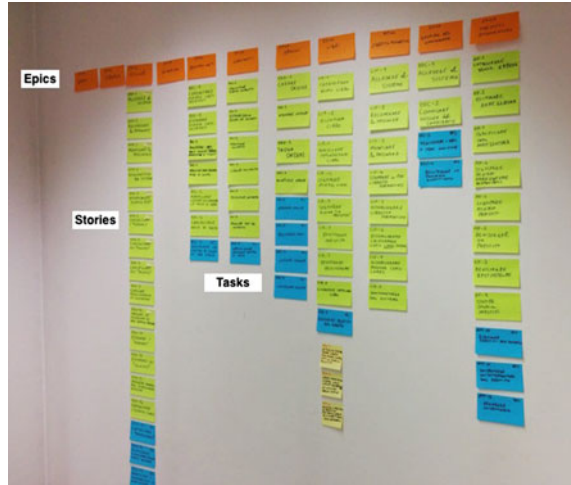
- if the story presents several variants of business rules, we can split the story in “main” and “secondary” rules.

It should be noted that the splitting activity is not all done up-front: it is an iterative activity. It is done at each iteration planning and in grooming sessions (see later on). We first split the story that gives the maximum value to the end user (the product owner voice here is essential) *and then* other big stories.

It should be noted that before starting a new iteration, we need a set of fine-grained user stories (allegedly those on the top of backlog); then, during the iteration, we spent time to clarify and reason more about other user stories. Indeed, we can deliver the maximum value to the end user in shortest possible time, so we start implementing a user story with a “just enough” understanding of it, investing other time on more examination when (and if) needed.

These “continuous” refinement activities on user stories are generally called “grooming”: the set of user stories—generally called “the product backlog”—has to be continually maintained in order to be ready as possible for next iterations. To manage this set of stories at a different level of detail without losing the big picture, we use a User Story Map [10]. This map is built incrementally and it represents a high-level view of activity done by the end user.

In our experience, the use of so-called low-fidelity tools such as post-it notes and whiteboards in managing user stories demonstrated a really effective way to (1) improve collaboration among people and (2) expose the whole set of features of a system/sub-system in a single, visible, place (Fig. 5).

Fig. 5 User story map

7 Conclusion

We choose to extensively use “user stories” in our development process for various reasons:

1. User stories avoid collecting too many details up-front, we delay such very moment when discussing “what” a user story really needs to be considered completed (and testable). This discussion arises when the development team considers including the given user story for the next iteration: until then we do not invest time and effort in gathering too many details, nor in estimating the *complexity size* of the story.
2. Although user stories represent, in our experience, an *effective* and *efficient* way to collect user needs, in some cases, we can—obviously—invest time in detailing some more elaborated “scenarios” when that helps avoid misinterpretations or helps clarifying system boundaries, constraints, and so on.
3. User stories carry the important “side effects” of the *need of a conversation* before considering its implementation. During this conversation(s), the development team can ask a wide range of questions to the business owner that represents the “voice of the end users.” This is an important, and too often underestimated, aspect: face-to-face conversations with the help of low-fidelity prototype (post-it, whiteboard) are way too effective in clarity and expressiveness respect to—for example—a long and detailed requirements document (see also [11] for more insight about that).
4. Having had experience in sectors particularly concerned with quality, assurance, and urgency of delivery (mainly banks and financial), we believe that the approaches described here can also be applied for defense field. To this end, the considerations expressed in [12] can be considered fully supported.

References

1. Schwaber K (2004) Agile project management with scrum. Microsoft Press, Washington, DC
2. Beck K (1999) Extreme programming explained: embrace change. Addison-Wesley, Boston
3. Cohn M (2004) User stories applied: for agile software development. Addison-Wesley Professional, Boston
4. Five Ws technique on information gathering. http://en.wikipedia.org/wiki/Five_Ws
5. Jeffries, R (2001) Essential XP: card, conversation, confirmation. <http://ronjeffries.com/xprog/articles/expcardconversationconfirmation>
6. Wake B (2003) Invest in good stories, and smart tasks. <http://xp123.com/xplor/xp0308/>
7. Beck K (2003) Test-driven development by example. Addison Wesley-Vaseem, Boston
8. Chrissis MB, Konrad M, Shrum S (2011) CMMI for development. guidelines for process integration and product improvement. Addison-Wesley, Boston
9. Jacobson I (1992) Object oriented software engineering: a use case driven approach. Addison-Wesley, Boston
10. Patton J (2014) User story mapping. O'Reilly, Newton
11. Cockburn A (2000) Writing effective use cases. Addison-Wesley Professional, Boston
12. Lapham, MA (2010) Considerations for using agile in DoD acquisition. TECHNICAL NOTE CMU/SEI-2010-TN-002. SEI